
Institut National des Postes et Télécommunications

INPT, Rabat

Année universitaire : 2025/2026

Mini Projet, ICCN INE1

Plateforme Collaborative

**Sécurisée pour le
Partage de Fichiers**

TRANSFERLY

Rapport, Étude Bibliographique

Première partie du rapport

Réalisé par, Groupe 4 :

Salma Douramane

Imane Elouahi

Nizar El Amrani

Zakaria Tahri

Jean-Arthur Awi-Détine

Encadré par :

Pr. Asmaa ElKandoussi

Pr. Charifa Hanin

Pr. Meryeme Ayache

Mars 2026

Table des matières

Introduction générale	1
1 Contexte et enjeux	2
1.1 Introduction	2
1.2 L'importance du partage de fichiers en milieu collaboratif	2
1.3 Les enjeux de sécurité	2
1.3.1 Confidentialité	3
1.3.2 Intégrité	3
1.3.3 Disponibilité	3
1.4 Présentation du projet	3
1.4.1 Fiche technique	3
1.4.2 Prototype de l'interface	3
1.5 Présentation des solutions existantes	5
1.5.1 Google Drive	6
1.5.2 Dropbox	6
1.5.3 Nextcloud	6
1.5.4 SharePoint (Microsoft)	6
1.6 Comparaison des solutions	7
1.7 Analyse des technologies et protocoles de sécurité	7
1.7.1 Protocoles de communication	7
1.7.2 Chiffrement des données stockées	7
1.7.3 Hachage des mots de passe	8
1.7.4 Authentification multi-facteurs (MFA)	8
1.7.5 Contrôle d'accès par ACL	8
1.7.6 Rôles utilisateurs et permissions	9
1.7.7 Protection contre les attaques courantes	10
1.8 Conclusion	10
2 Conception et architecture de la solution	11
2.1 Introduction	11
2.2 Modélisation UML	11
2.2.1 Diagramme de cas d'utilisation	11
2.2.2 Diagramme de classes	13
2.2.3 Diagrammes de séquence	13

2.2.4	Diagrammes d'activité	16
2.3	Architecture de la solution	18
2.3.1	Vision globale	18
2.3.2	Architecture physique — Deux VMs	19
2.3.3	Architecture logique — Three-Tier	20
2.3.4	Architecture physique et déploiement	22
2.3.5	Modèle de sécurité, Défense en profondeur	22
2.4	Technologies retenues	23
2.5	Conclusion	23
Bibliographie		23

Introduction générale

Le partage de fichiers est devenu, ces dernières années, une pratique quotidienne dans pratiquement tous les environnements professionnels et éducatifs. Que ce soit pour travailler en équipe sur un projet, transmettre des documents entre collègues ou centraliser des ressources communes, le besoin d'une plateforme fiable et accessible est constant.

Pourtant, cette facilité de partage soulève une question qu'on a tendance à sous-estimer : celle de la sécurité. Dans un réseau local ou un environnement collaboratif, laisser les fichiers accessibles à tout le monde sans contrôle précis peut rapidement poser des problèmes : accès non autorisé à des données sensibles, modifications involontaires, suppression accidentelle, etc. Il ne suffit pas que l'outil soit pratique, il doit aussi être sûr.

C'est dans ce cadre que s'inscrit notre mini-projet. L'objectif est de concevoir et développer **Transferly**, une plateforme web collaborative permettant de partager des fichiers de manière sécurisée, avec un système de gestion des utilisateurs, des permissions fines et une traçabilité complète des actions.

Ce rapport constitue la première partie de notre travail : une étude bibliographique visant à explorer le contexte du projet, analyser les solutions existantes sur le marché et poser les bases de notre architecture technique.

Nous commençons par situer le projet dans son contexte (Chapitre 1), puis nous présentons la conception et l'architecture que nous avons retenues (Chapitre 2).

Chapitre 1

Contexte et enjeux

1.1 Introduction

Avant de se lancer dans le développement d'une plateforme, il est utile de comprendre pourquoi elle est nécessaire. Ce chapitre présente le contexte général du partage de fichiers en environnement collaboratif, les défis de sécurité que cela implique, puis un tour d'horizon des solutions existantes et des technologies qui les sous-tendent.

1.2 L'importance du partage de fichiers en milieu collaboratif

Dans une entreprise ou une institution éducative, les données circulent en permanence. Les équipes partagent des rapports, des présentations, du code source, des bases de données internes et ce flux ne cesse d'augmenter avec la généralisation du télétravail et des organisations distribuées. Selon une étude IDC de 2022, un employé passe en moyenne plus de deux heures par jour à chercher ou à partager des documents.

Dans ce contexte, disposer d'un espace commun structuré n'est plus un luxe, c'est une nécessité opérationnelle. Un bon système de partage de fichiers doit permettre :

- d'accéder aux ressources rapidement, depuis n'importe quel poste du réseau ;
- de collaborer sur les mêmes fichiers sans écraser le travail des autres ;
- de garder une trace des modifications effectuées ;
- et surtout, de s'assurer que chaque utilisateur n'accède qu'à ce qu'il est censé voir.

Ce dernier point est souvent négligé dans les environnements moins formels, où on utilise un simple dossier partagé sur le réseau sans aucune gestion des droits. Les risques sont réels, et c'est précisément là qu'intervient la notion de sécurité.

1.3 Les enjeux de sécurité

La sécurité d'une plateforme de partage repose essentiellement sur trois piliers classiques : la **confidentialité**, l'**intégrité** et la **disponibilité**, ce qu'on résume généralement sous le sigle CIA (Confidentiality, Integrity, Availability).

1.3.1 Confidentialité

Il faut garantir que seules les personnes autorisées peuvent accéder à un fichier donné. Un utilisateur ne devrait même pas être capable de voir l'existence d'un fichier auquel il n'a pas accès. Cela implique un système de contrôle d'accès rigoureux, couplé à un mécanisme d'authentification solide.

1.3.2 Intégrité

Les données ne doivent pas être modifiées sans autorisation. En cas de modification, le système doit être en mesure de le détecter et, idéalement, de revenir à une version précédente. C'est le rôle du versionnement des fichiers.

1.3.3 Disponibilité

Le service doit rester accessible, même en cas d'accès simultanés importants. La gestion des accès concurrents est donc un point technique non négligeable dans la conception.

À ces trois piliers, on peut ajouter la **traçabilité** : enregistrer qui a fait quoi et quand. En cas d'incident, les journaux d'activité (logs) permettent de reconstituer les événements et d'identifier la source du problème.

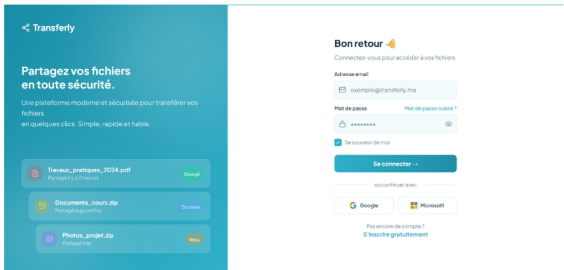
1.4 Présentation du projet

1.4.1 Fiche technique

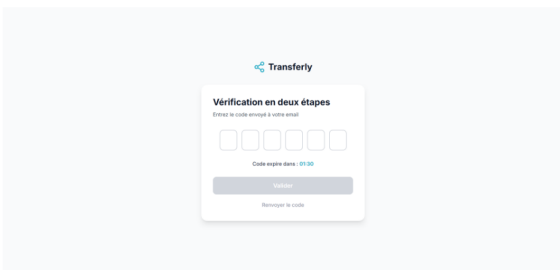
Nom du projet	Transferly
Type	Plateforme web de partage de fichiers sécurisée
Contexte	Mini-projet ICCN INE1, INPT 2025/2026
Groupe	Groupe 4
Technologies	Python/Flask, React, SQLite
Fonctionnalités principales	Authentification MFA, gestion des rôles, ACL, versionnement, journalisation
Déploiement	Local dans un premier temps, puis réseau inter-équipes

1.4.2 Prototype de l'interface

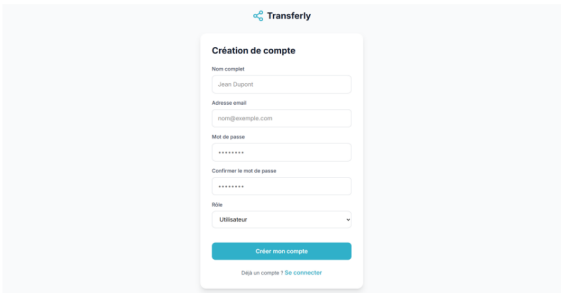
Un prototype complet de l'interface a été réalisé sur Figma avant le développement. Il couvre l'ensemble des pages de la plateforme et a permis de valider nos choix de conception auprès de l'équipe avant de passer au code.



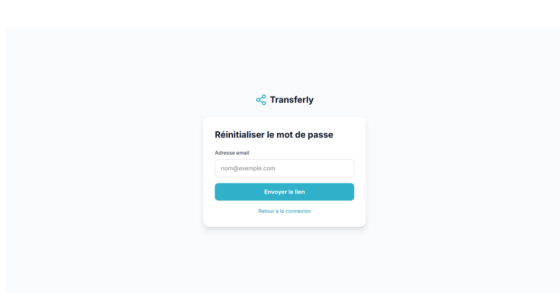
(a). Page d'authentification



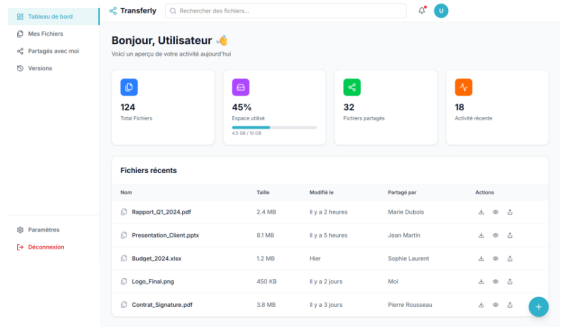
(b). Double authentication (MFA)



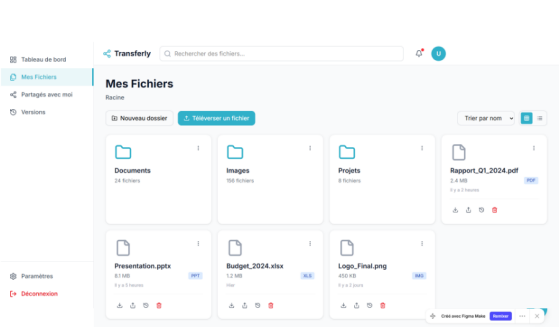
(a). Création de compte



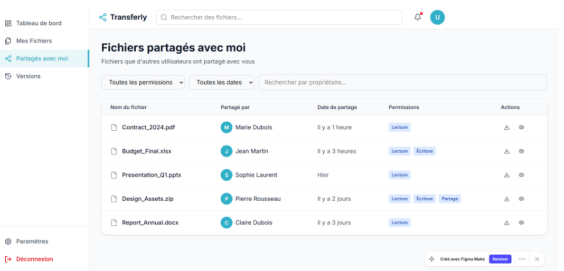
(b). Récupération du mot de passe



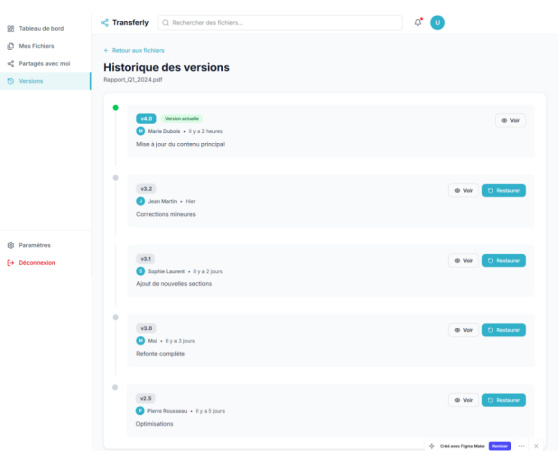
(a). Tableau de bord utilisateur



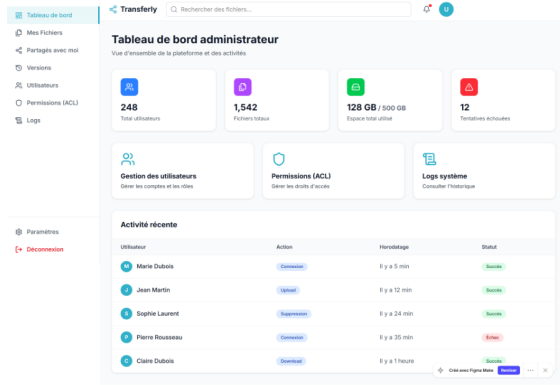
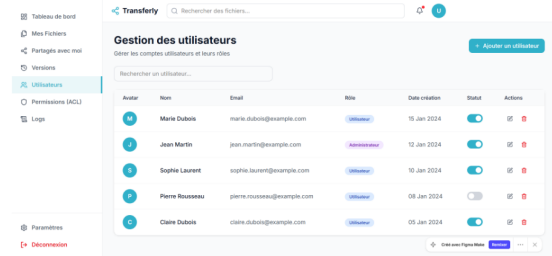
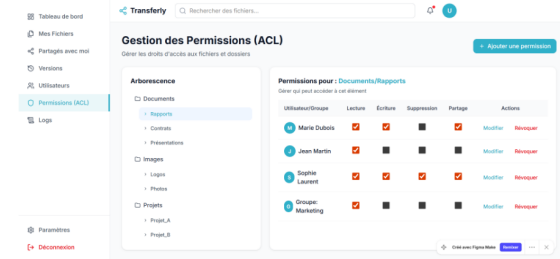
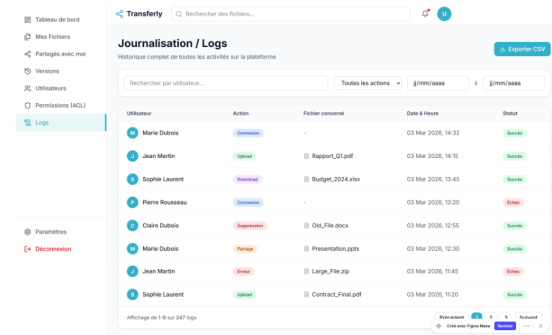
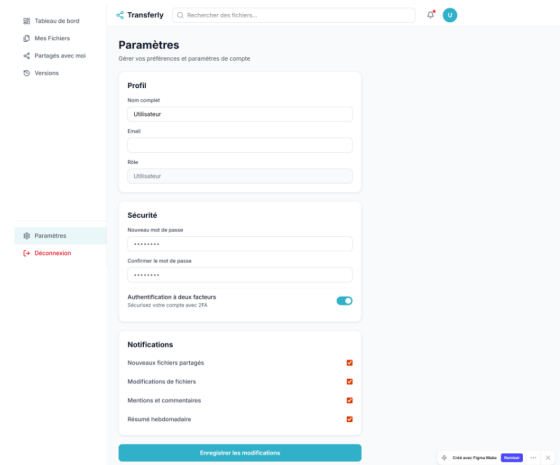
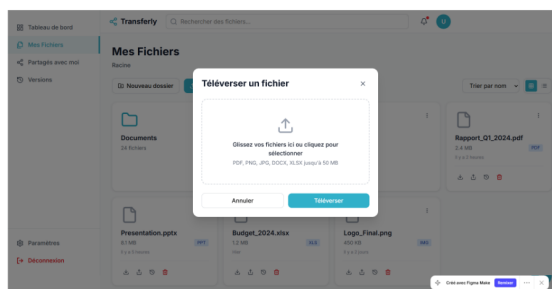
(b). Fichiers personnels



(a). Fichiers partagés avec moi



(b). Historique des versions

(a). *Tableau de bord administrateur*(b). *Gestion des utilisateurs*(a). *Gestion des permissions ACL*(b). *Journalisation et logs système*(a). *Paramètres utilisateur*(b). *Téléversement d'un fichier*

Prototype de l'interface Transferly (réalisé sur Figma)

1.5 Présentation des solutions existantes

Avant de développer notre propre plateforme, il était logique de regarder ce qui existe déjà. Plusieurs solutions de partage de fichiers sont largement utilisées aujourd'hui.

1.5.1 Google Drive

Google Drive est probablement la solution la plus connue du grand public. Elle offre un espace de stockage en ligne avec des fonctionnalités de collaboration en temps réel (Google Docs, Sheets, etc.). La gestion des partages est simple et intuitive, mais les données sont hébergées sur des serveurs Google, ce qui pose des questions de confidentialité, notamment pour des données sensibles d'entreprise.

1.5.2 Dropbox

Dropbox est historiquement l'un des pionniers du stockage cloud. Il propose une synchronisation automatique des fichiers entre appareils et un partage par lien. Les fonctionnalités de contrôle d'accès restent cependant assez basiques dans les versions standards, et la gestion fine des permissions nécessite un abonnement Business.

1.5.3 Nextcloud

Nextcloud est une solution open source auto-hébergée qui se rapproche davantage de notre projet. Elle permet de déployer son propre serveur de fichiers avec gestion des utilisateurs, des groupes, des permissions et un historique des versions. C'est probablement la référence la plus pertinente pour notre contexte, même si sa complexité de déploiement dépasse largement le cadre d'un mini-projet académique.

1.5.4 SharePoint (Microsoft)

SharePoint est la solution de Microsoft pour les entreprises. Très puissant, il s'intègre avec l'ensemble de l'écosystème Microsoft 365. Son principal inconvénient est sa lourdeur : la configuration est complexe et nécessite des compétences spécifiques. Il est davantage adapté aux grandes organisations qu'à un environnement de réseau local académique.

1.6 Comparaison des solutions

Critère	Google Drive	Dropbox	Nextcloud	Transferly
Hébergement	Cloud (Google)	Cloud	Auto-hébergé	Local / LAN
Authentification MFA	Oui	Oui	Oui	Oui
Contrôle ACL fin	Limité	Limité	Oui	Oui
Versionnement	Oui	Oui	Oui	Oui
Journalisation	Partielle	Partielle	Complète	Complète
Open source	Non	Non	Oui	Oui (projet)
Adapté LAN académique	Non	Non	Partiellement	Oui

Ce tableau met en évidence que les solutions cloud grand public ne sont pas vraiment adaptées à un usage en réseau local contrôlé. Nextcloud est la plus proche de nos besoins, mais sa complexité justifie le développement d’une solution propre, taillée aux exigences de notre cahier des charges.

1.7 Analyse des technologies et protocoles de sécurité

1.7.1 Protocoles de communication

La communication entre le navigateur et notre serveur repose sur **HTTP/HTTPS**. Le protocole **TLS** (Transport Layer Security) assure le chiffrement des échanges en transit et empêche qu’un attaquant interceptant le trafic réseau puisse lire les données.

1.7.2 Chiffrement des données stockées

Tout fichier téléversé sur la plateforme est **automatiquement chiffré au moment de son enregistrement** sur le serveur, sans intervention de l’utilisateur. Ce chiffrement est réalisé via **AES-128 en mode CBC** à travers l’implémentation **Fernet** de la bibliothèque Python `cryptography`.

Ce choix est justifié par plusieurs raisons complémentaires :

- **Standardisation NIST.** L’AES (Advanced Encryption Standard) est le standard symétrique recommandé par le NIST depuis 2001 (FIPS 197). Son utilisation est universellement reconnue dans les environnements à contraintes de sécurité.

- **Intégrité garantie.** Fernet intègre nativement un mécanisme d'authentification HMAC-SHA256 : si un fichier stocké est altéré (corruption, tentative de falsification), le déchiffrement échoue et l'intégrité de la donnée est vérifiable sans outil externe.
- **Facilité d'intégration.** La bibliothèque `cryptography` offre une API de haut niveau qui réduit les risques d'erreurs d'implémentation, fréquentes lorsqu'on manipule directement des primitives cryptographiques (gestion du vecteur d'initialisation, padding, etc.).
- **Protection au repos.** Même si un attaquant accède physiquement au système de fichiers du serveur, il ne peut pas lire le contenu des fichiers sans la clé symétrique, stockée séparément et protégée par les permissions du système d'exploitation.

La clé de chiffrement est générée à l'initialisation du serveur et stockée dans un fichier `.key` protégé, accessible uniquement par le processus Flask. Les métadonnées (nom, taille, propriétaire) restent en clair en base de données pour permettre la navigation, tandis que le contenu binaire des fichiers est systématiquement chiffré.

1.7.3 Hachage des mots de passe

Les mots de passe ne sont jamais stockés en clair. On utilise **Bcrypt**, un algorithme de hachage spécialement conçu pour les mots de passe, intentionnellement lent pour rendre les attaques par force brute coûteuses. C'est une recommandation standard de l'OWASP.

1.7.4 Authentification multi-facteurs (MFA)

En plus du mot de passe, l'accès à la plateforme nécessite la validation d'un code OTP (One-Time Password) généré via PyOTP, envoyé par email ou via une application d'authentification.

1.7.5 Contrôle d'accès par ACL

Les ACL (Access Control Lists) définissent pour chaque fichier ou dossier quels utilisateurs disposent des droits suivants :

- **Lecture** : consulter et visualiser le contenu du fichier ;
- **Écriture** : modifier le contenu d'un fichier existant ;
- **Upload** : déposer de nouveaux fichiers dans un espace ou un dossier ;
- **Download** : télécharger le fichier vers son poste local ;
- **Suppression** : effacer définitivement un fichier ou un dossier ;

- **Partage** : accorder à d'autres utilisateurs des droits d'accès sur une ressource dont on est propriétaire ou administrateur.

Ces permissions sont indépendantes et combinables : un utilisateur peut par exemple détenir le droit de lecture et de téléchargement sans pouvoir modifier ni supprimer. Elles sont stockées en base de données et vérifiées à chaque requête par le moteur ACL du backend.

1.7.6 Rôles utilisateurs et permissions

La plateforme distingue trois rôles principaux, chacun associé à un périmètre de permissions précis.

Administrateur Global (AdminGlobal)

L'AdminGlobal dispose d'un accès complet à l'ensemble du système. Il peut :

- créer, modifier et supprimer tout compte utilisateur ;
- attribuer et révoquer les rôles AdminEspace et Utilisateur standard ;
- consulter et exporter les journaux d'activité (logs) complets ;
- modifier le quota de stockage de n'importe quel utilisateur (valeur par défaut : 10 Go, modifiable individuellement) ;
- accéder à tous les espaces et fichiers de la plateforme, indépendamment des ACL définies par les administrateurs d'espace ;
- configurer les paramètres globaux du système (durée de rétention des versions, politique de mots de passe, etc.).

Administrateur d'espace (AdminEspace)

L'AdminEspace gère un ou plusieurs espaces collaboratifs qui lui sont attribués par l'AdminGlobal. Dans son périmètre, il peut :

- inviter des utilisateurs dans son espace et leur retirer l'accès ;
- définir et modifier les règles ACL (Lecture, Écriture, Upload, Download, Suppression, Partage) pour chaque fichier ou dossier de son espace ;
- consulter les journaux d'activité relatifs à son espace uniquement ;
- restaurer des versions antérieures de fichiers dans son espace ;
- visualiser l'occupation du quota de chaque membre de son espace.

Il n'a pas accès aux espaces d'autres AdminEspace et ne peut pas modifier les paramètres système ni les quotas.

Utilisateur standard

L'utilisateur standard interagit avec les fichiers selon les permissions ACL qui lui ont été accordées. Par défaut, il peut :

- téléverser, télécharger, lire et modifier les fichiers pour lesquels il dispose des droits correspondants ;
- partager ses propres fichiers avec d'autres utilisateurs, dans la limite du droit de Partage accordé par l'AdminEspace ;
- consulter l'historique des versions de ses fichiers ;
- visualiser depuis son tableau de bord l'espace de stockage consommé et le quota total qui lui est alloué ;
- modifier ses informations personnelles et ses préférences de notification.

Il ne peut pas voir les fichiers d'autres utilisateurs auxquels il n'a pas accès, ni consulter les journaux système.

1.7.7 Protection contre les attaques courantes

- **Limitation des tentatives de connexion** : protection contre le brute force ;
- **Validation des fichiers téléversés** : vérification du type MIME et de l'extension ;
- **Filtrage visuel** : les fichiers non autorisés n'apparaissent pas dans l'interface.

1.8 Conclusion

Ce premier chapitre nous a permis de comprendre pourquoi la sécurité est un enjeu central dans toute plateforme de partage de fichiers, et de constater que les solutions existantes ne répondent pas parfaitement aux besoins d'un environnement local académique. Le chapitre suivant décrit la conception et l'architecture retenues pour Transferly.

Chapitre 2

Conception et architecture de la solution

2.1 Introduction

Ce chapitre présente la conception technique de Transferly à travers la modélisation UML du système, puis l'architecture logicielle et physique retenue.

2.2 Modélisation UML

2.2.1 Diagramme de cas d'utilisation

Le diagramme ci-dessous identifie les trois acteurs principaux de la plateforme : l'utilisateur standard, l'administrateur d'espace et l'administrateur global, ainsi que l'ensemble des fonctionnalités auxquelles chacun a accès.

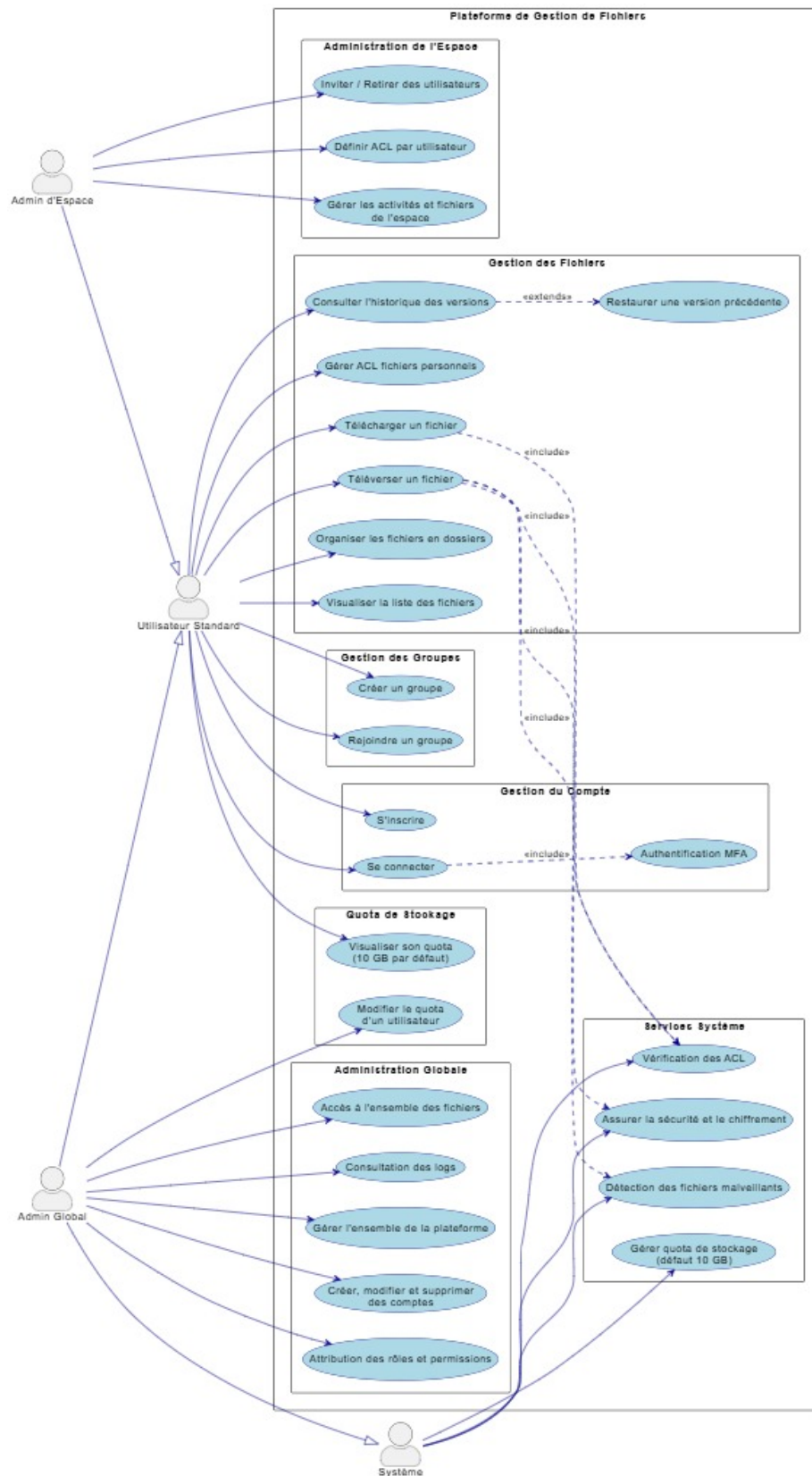


Figure 2.1. Diagramme de cas d'utilisation, Plateforme Transferly

2.2.2 Diagramme de classes

Le diagramme de classes décrit la structure statique du système : les entités principales, leurs attributs, leurs méthodes et les relations entre elles (association, héritage, composition).

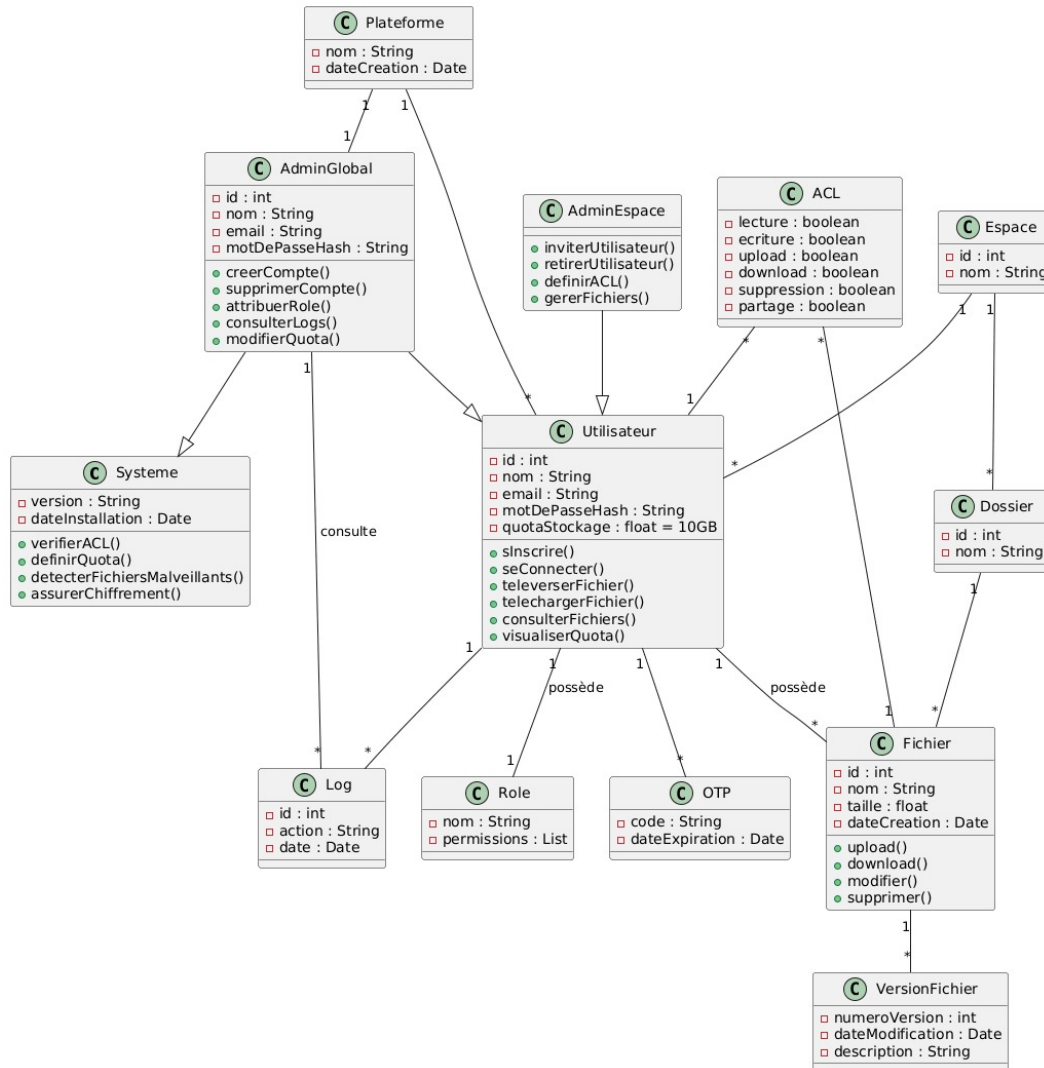


Figure 2.2. Diagramme de classes, Structure statique du système

2.2.3 Diagrammes de séquence

Les diagrammes de séquence illustrent les échanges dynamiques entre les composants du système pour les scénarios critiques de la plateforme.

1.3.1 Inscription

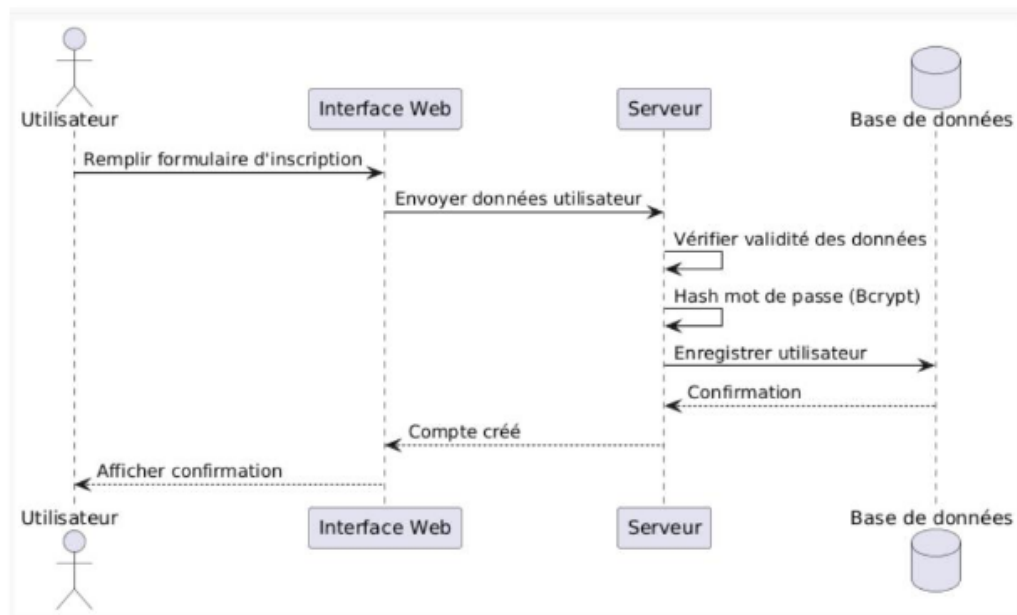


Figure 2.3. Diagramme de séquence, Inscription d'un utilisateur

L'utilisateur remplit le formulaire d'inscription. Le serveur vérifie la validité des données, hache le mot de passe via Bcrypt, enregistre l'utilisateur en base de données et renvoie une confirmation à l'interface.

1.3.2 Connexion avec MFA

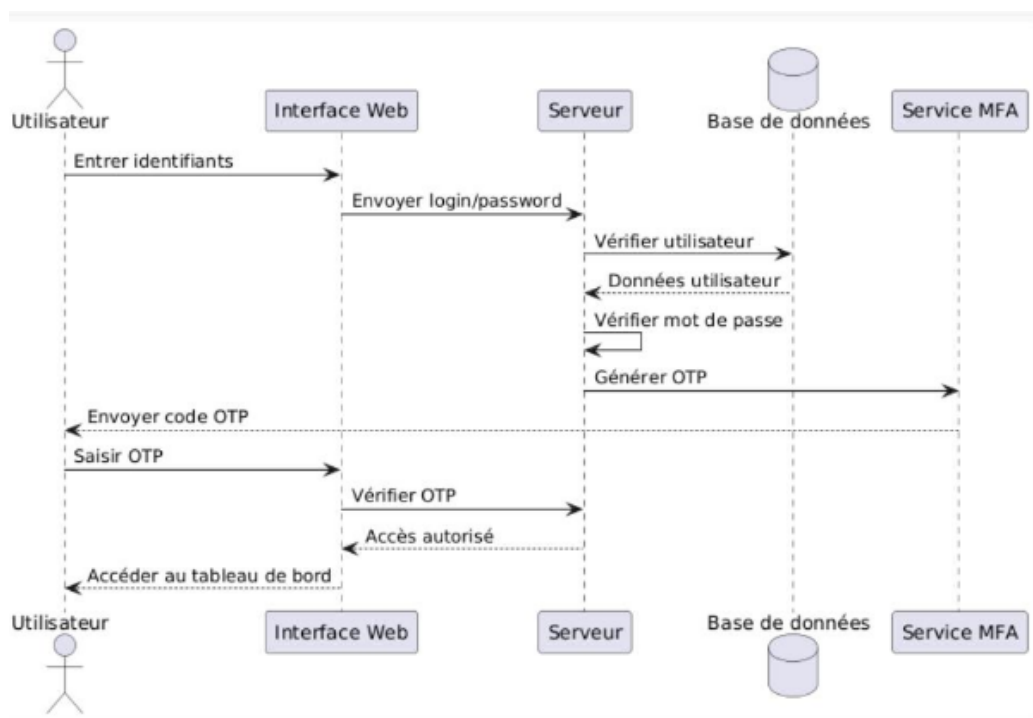


Figure 2.4. Diagramme de séquence, Connexion avec authentification MFA

L'utilisateur saisit ses identifiants. Si ceux-ci sont valides, un code OTP est généré et envoyé. L'utilisateur soumet ce code, qui est vérifié côté serveur avant d'autoriser l'accès au tableau de bord.

1.3.3 Gestion des ACL

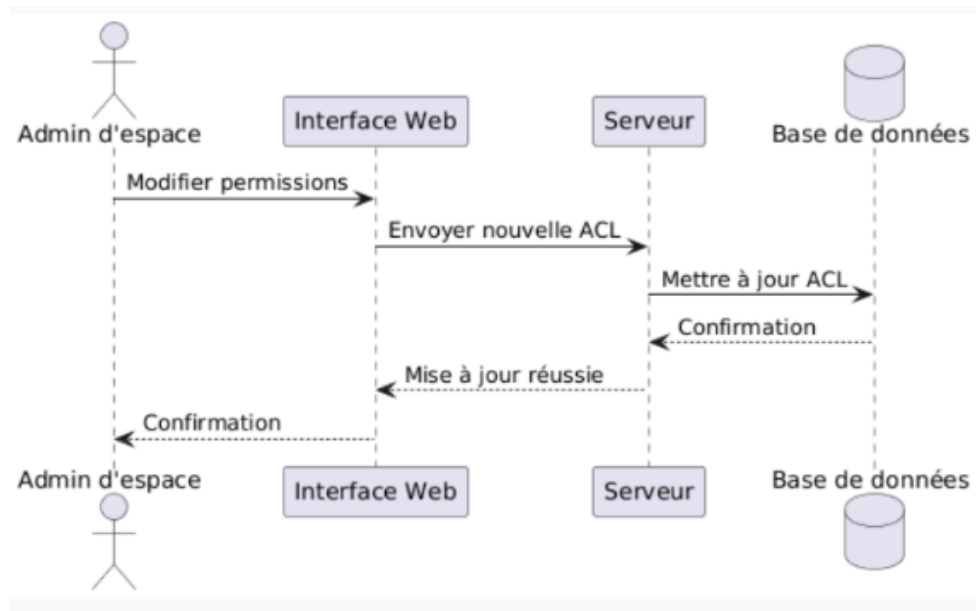


Figure 2.5. Diagramme de séquence, Modification des permissions ACL

L'administrateur d'espace envoie une nouvelle règle ACL via l'interface. Le serveur met à jour l'entrée correspondante en base de données et confirme la mise à jour.

1.3.4 Téléversement d'un fichier

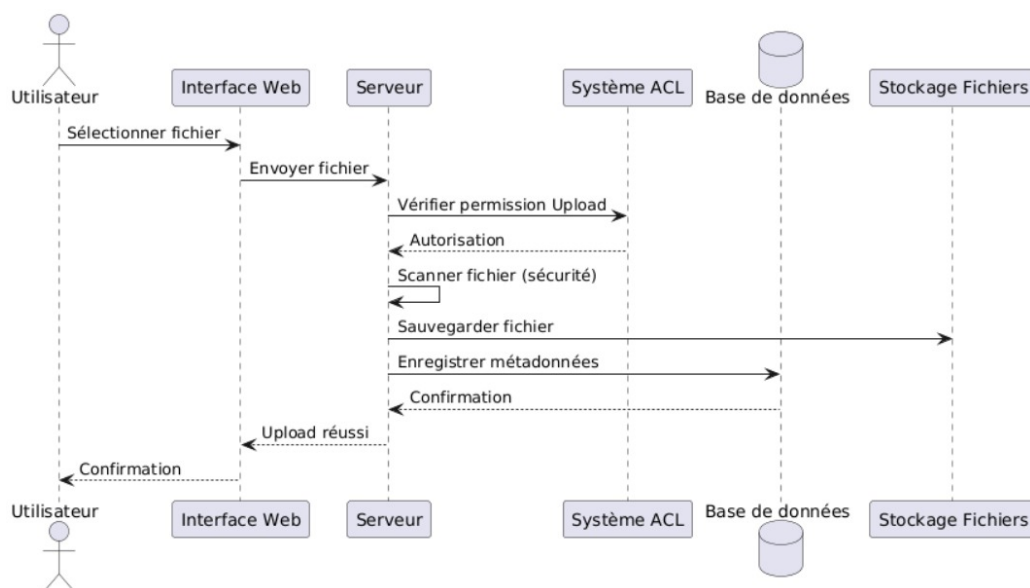


Figure 2.6. Diagramme de séquence, Téléversement (upload) d'un fichier

Avant tout stockage, le serveur vérifie la permission d'écriture via le système ACL, puis scanne le fichier (type MIME, extension). Si tout est correct, le fichier est sauvegardé et les métadonnées enregistrées en base.

1.3.5 Téléchargement d'un fichier

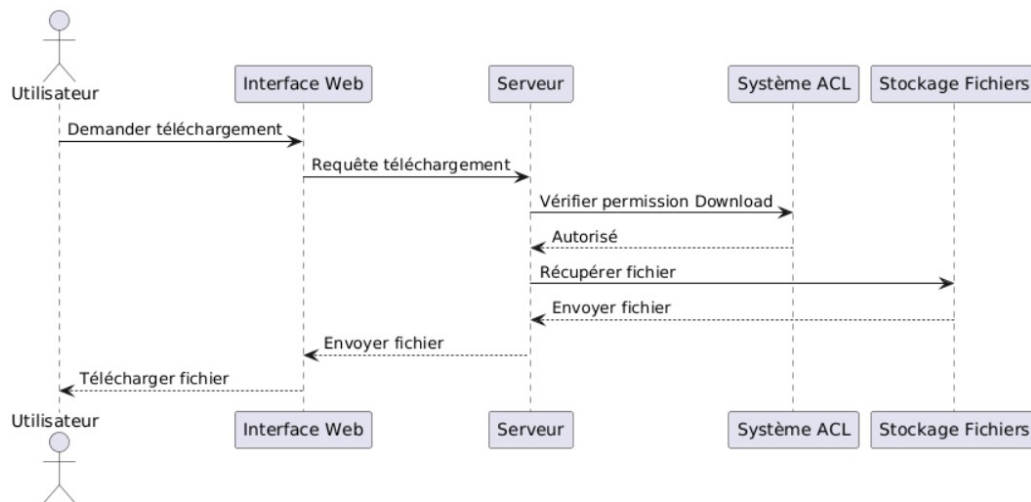


Figure 2.7. Diagramme de séquence, Téléchargement (download) d'un fichier

Le serveur vérifie la permission de lecture ACL avant de récupérer le fichier depuis le stockage et de l'envoyer au client. Toute tentative refusée est également enregistrée dans les logs.

2.2.4 Diagrammes d'activité

Les diagrammes d'activité modélisent le flux de contrôle des processus métier principaux, en mettant en évidence les décisions et les actions parallèles.

1.4.1 Authentification avec MFA

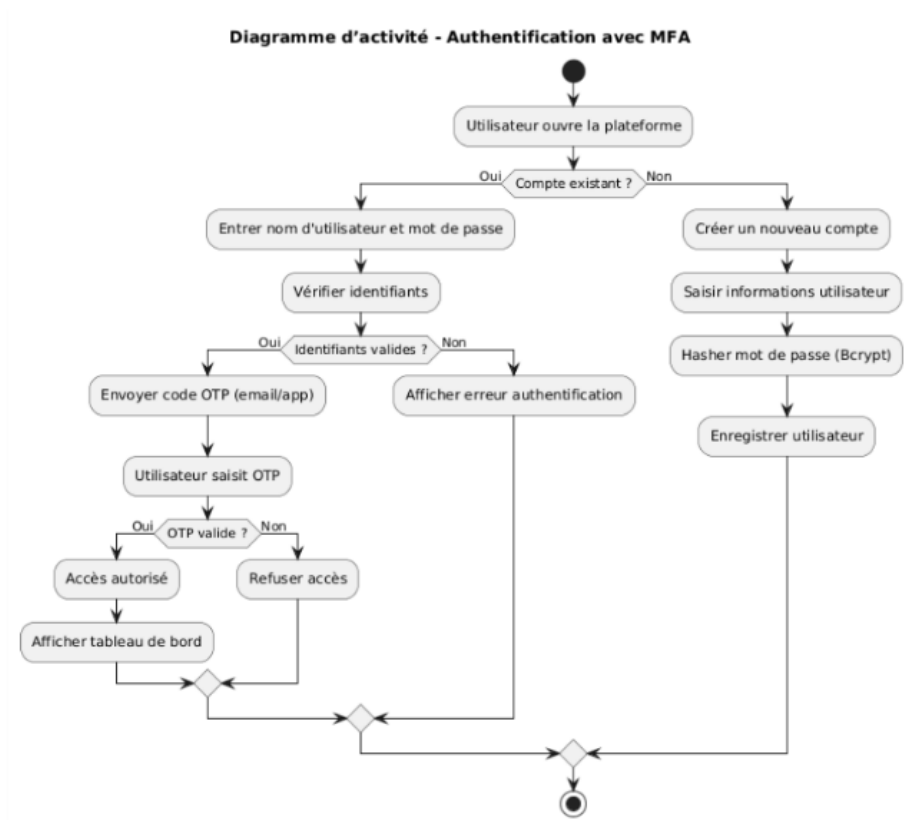


Figure 2.8. *Diagramme d'activité, Authentification avec MFA*

Le diagramme modélise deux chemins : si l'utilisateur possède un compte, il entre ses identifiants et passe par la validation OTP. Dans le cas contraire, il crée un compte, son mot de passe est haché et il est enregistré en base.

1.4.2 Gestion des fichiers

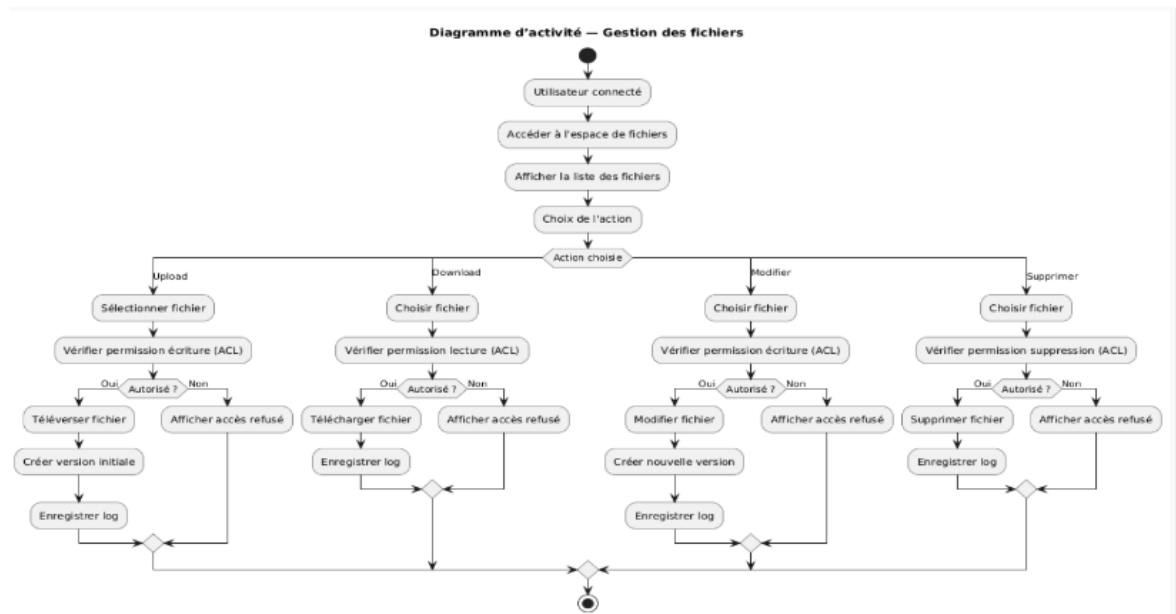


Figure 2.9. Diagramme d'activité, Gestion des fichiers (upload, download, modification, suppression)

Une fois connecté, l'utilisateur choisit une action parmi upload, download, modification ou suppression. Chaque action déclenche une vérification ACL correspondante. Si l'autorisation est accordée, l'action est exécutée et un log est enregistré ; sinon, un message d'accès refusé est affiché.

2.3 Architecture de la solution

2.3.1 Vision globale

Transferly est conçue comme un serveur de fichiers collaboratif accessible via un navigateur web. Elle centralise la gestion des fichiers, des utilisateurs et des permissions au sein d'un même système. Le déploiement repose sur **deux machines virtuelles (VMs) distinctes**, séparées par une frontière réseau : le frontend est exposé dans une **DMZ** (zone démilitarisée), tandis que le backend réside dans une **zone interne sécurisée**, inaccessible directement depuis l'extérieur.

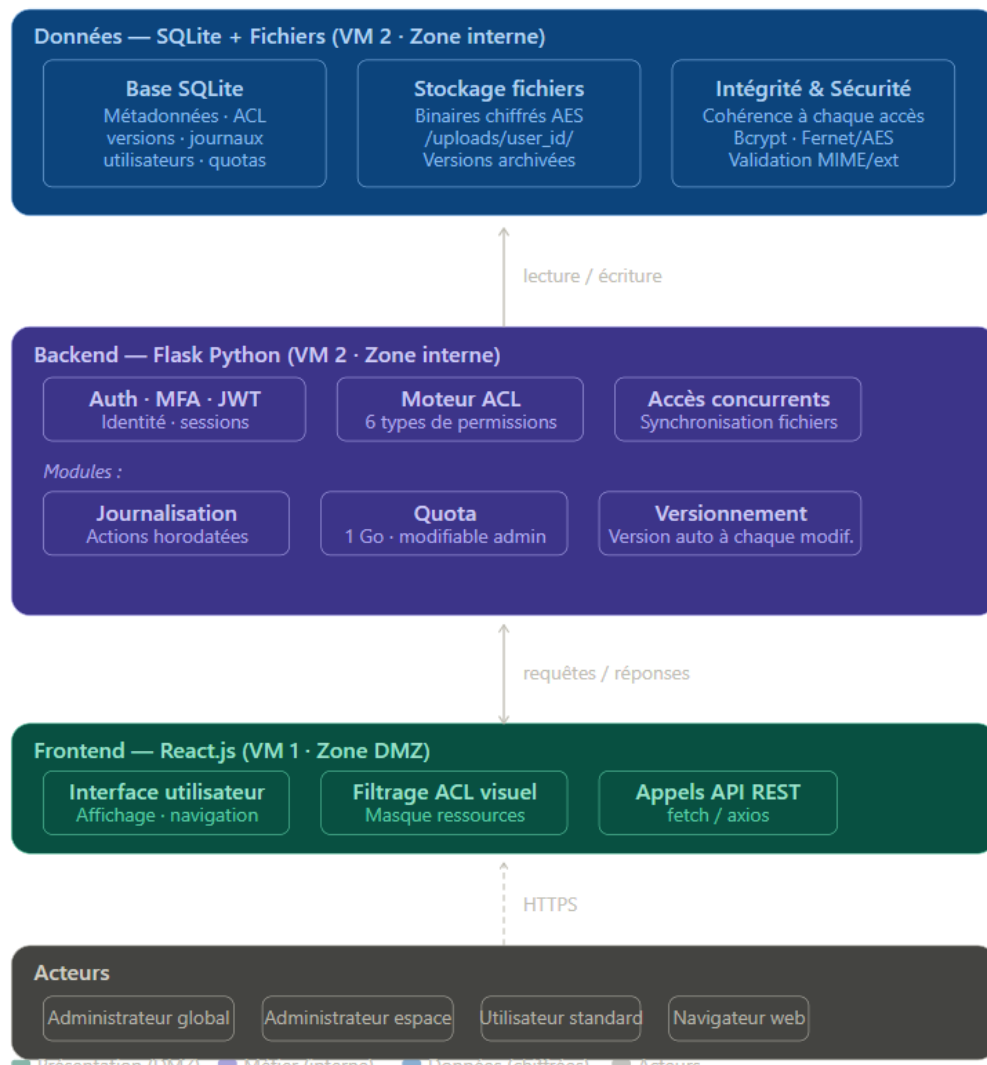


Figure 2.10. Architecture globale du système Transferly

Toute requête d'accès à un fichier passe obligatoirement par le moteur ACL avant exécution, et chaque action est systématiquement enregistrée dans les journaux.

2.3.2 Architecture physique — Deux VMs

VM 1 — Frontend (DMZ). Cette première machine virtuelle héberge l'interface React, servie via un serveur web (Nginx). Elle est accessible depuis les postes clients du réseau académique. Son exposition en DMZ signifie qu'elle est isolée du reste du système par des règles de pare-feu strictes : elle peut uniquement communiquer avec la VM 2 (backend) sur un port dédié à l'API REST. En cas de compromission de la VM 1, le backend et la base de données restent protégés par la segmentation réseau.

VM 2 — Backend (zone interne). La seconde machine virtuelle héberge le serveur Flask, la base de données SQLite et le stockage des fichiers chiffrés. Elle n'est *jamais*

exposée directement au réseau client : seule la VM 1 peut lui adresser des requêtes, via l'API interne. Cette isolation constitue la principale barrière de défense contre les attaques directes sur les données.

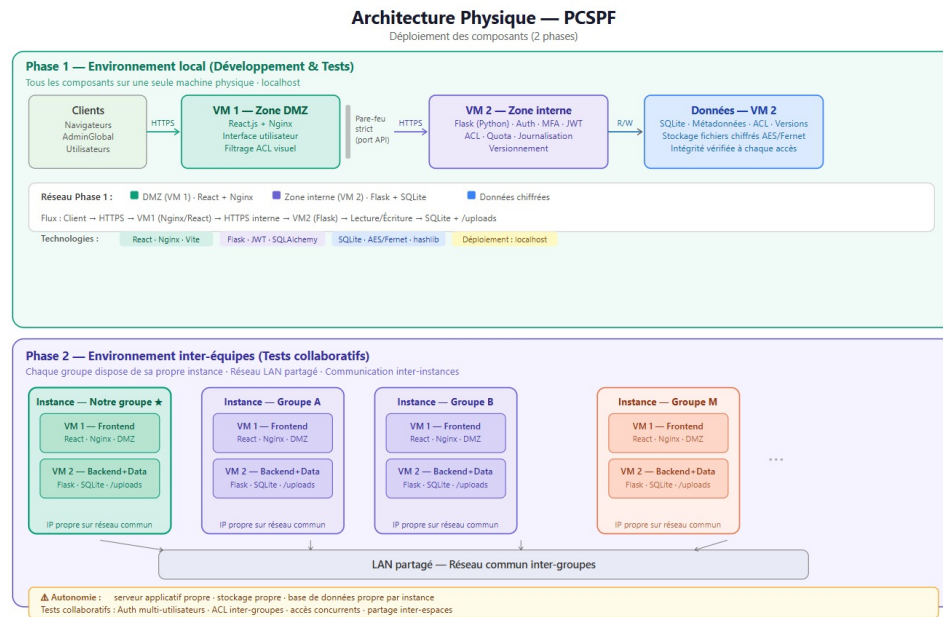


Figure 2.11. Architecture physique et segmentation réseau (DMZ vs Zone Interne)

Communication inter-VMs. Les échanges entre les deux VMs transitent sur un réseau interne dédié (VLAN privé). L'API REST exposée par le backend est sécurisée par TLS et par des tokens JWT à durée de vie limitée.

2.3.3 Architecture logique — Three-Tier

L'architecture logique repose sur le modèle classique en trois couches, réparties sur les deux VMs :

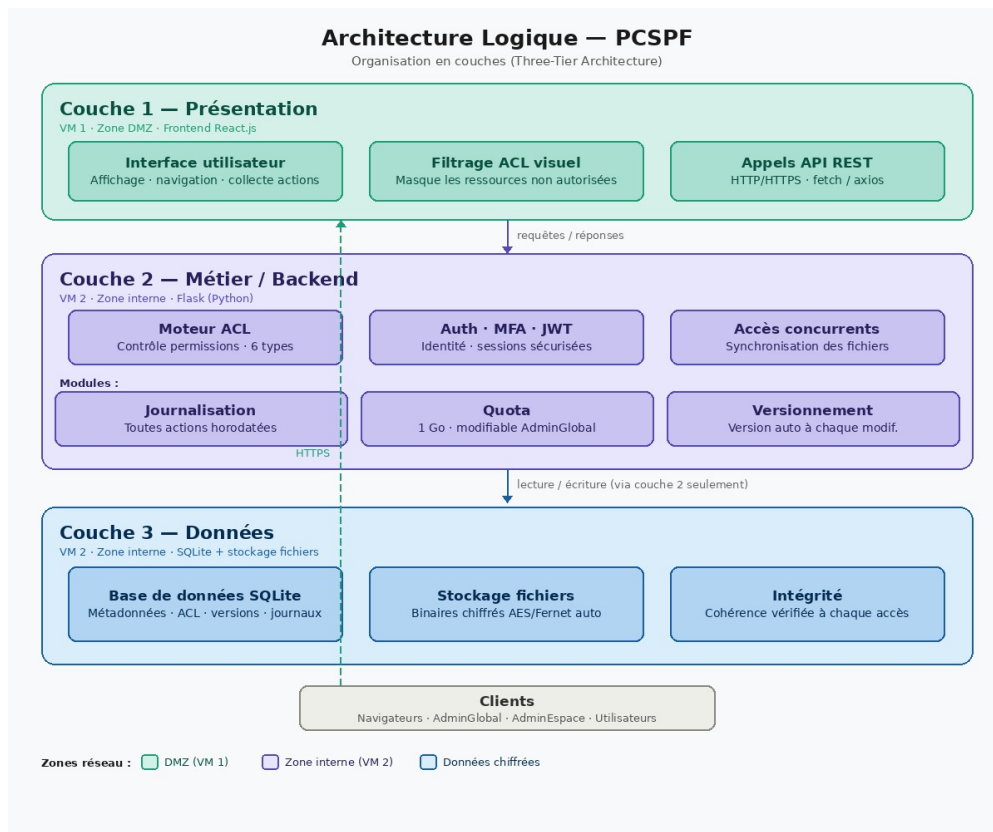


Figure 2.12. Architecture logique Three-Tier de la plateforme

Couche présentation (Frontend, React — VM 1) Interface utilisateur accessible via le navigateur. Elle affiche les données reçues et transmet les actions de l'utilisateur au backend via l'API REST. Elle ne contient aucune logique métier : un fichier non autorisé n'y apparaît tout simplement pas.

Couche métier (Backend, Flask — VM 2) C'est le cœur du système. Il se décompose en trois modules fonctionnels principaux :

- **Module de journalisation.** Enregistre de manière exhaustive toutes les actions réalisées sur la plateforme : connexions, téléversements, téléchargements, modifications de permissions, tentatives d'accès refusées, etc. Chaque entrée de log contient l'identifiant de l'utilisateur, le type d'action, l'horodatage et le statut (succès ou échec). Les journaux sont consultables uniquement par l'AdminGlobal, ou par l'AdminEspace pour son propre périmètre.
- **Module de quota.** Contrôle l'espace de stockage alloué à chaque utilisateur. Le quota par défaut est fixé à **2 Go par utilisateur**. L'AdminGlobal peut modifier cette valeur individuellement depuis l'interface d'administration. Avant chaque téléversement, le module vérifie que l'utilisateur dispose d'un espace suffisant ; si le quota est dépassé, l'opération est refusée et l'utilisateur est informé. Chaque utilisateur peut visualiser en temps réel l'espace consommé et le quota alloué depuis son tableau de bord personnel.

- **Module de versionnement.** À chaque modification d'un fichier existant, une nouvelle version est automatiquement créée et conservée. Les métadonnées de chaque version (numéro, date, auteur, taille, empreinte SHA-256) sont stockées en base de données. Les fichiers des versions antérieures sont conservés sur le disque sous forme chiffrée. L'AdminEspace et les utilisateurs autorisés peuvent consulter l'historique des versions et restaurer une version antérieure en un clic.

Couche données (SQLite et stockage fichiers — VM 2) Responsable du stockage et de la récupération des données. Elle gère les métadonnées structurées (comptes, rôles, ACL, versions, logs, quotas) en base SQLite, et les fichiers binaires chiffrés sur le système de fichiers local, organisés par utilisateur.

2.3.4 Architecture physique et déploiement

Phase 1 — Environnement local. Les deux VMs tournent sur le même hôte physique (hyperviseur VirtualBox ou KVM), reliées par un réseau interne virtuel. Cette configuration permet de développer et tester toutes les fonctionnalités, y compris la séparation DMZ/zone interne, dans un environnement entièrement maîtrisé.

Phase 2 — Réseau inter-équipes. Une fois le fonctionnement local validé, chaque groupe déploie ses instances sur un réseau local commun pour réaliser des tests collaboratifs : authentification multi-utilisateurs, ACL entre groupes, gestion des accès concurrents et partage inter-espaces.

2.3.5 Modèle de sécurité, Défense en profondeur

Notre approche repose sur plusieurs couches de protection indépendantes :

1. Hachage Bcrypt des mots de passe, aucun mot de passe en clair en base ;
2. Authentification MFA obligatoire, second facteur via OTP ;
3. Limitation du nombre de tentatives, protection contre le brute force ;
4. Vérification ACL à chaque requête, sans exception ;
5. Chiffrement AES des fichiers confidentiels au stockage ;
6. Validation du type et du contenu des fichiers téléversés ;
7. Journalisation complète de toutes les actions, consultable uniquement par l'administrateur global.

2.4 Technologies retenues

Composant	Technologie	Justification
Backend	Python / Flask	Légèreté, richesse des extensions, maîtrise de l'équipe
Frontend	React.js	Architecture par composants, gestion d'état dynamique
Base de données	SQLite	Sans serveur, intégration native Python, adapté au projet
Hachage MDP	Bcrypt	Standard OWASP, résistant au brute force
Chiffrement	AES / Fernet	Standard NIST, performant et robuste
MFA	PyOTP	Compatible TOTP/HOTP, simple à intégrer
Transfert fichiers	HTTP multipart	Standard web, natif Flask

2.5 Conclusion

Ce chapitre a présenté l'ensemble de la conception technique de Transferly, depuis la modélisation UML jusqu'à l'architecture physique et le modèle de sécurité. Les choix effectués (architecture en trois couches, défense en profondeur, technologies légères et maîtrisées) sont cohérents avec les contraintes d'un projet académique tout en respectant les bonnes pratiques du domaine.

Bibliographie

- [1] Pallets Projects. *Flask Documentation (3.x)*. Disponible sur : <https://flask.palletsprojects.com>.
- [2] Meta Open Source. *React, A JavaScript library for building user interfaces*. Disponible sur : <https://react.dev>.
- [3] Python Software Foundation. *sqlite3, DB-API 2.0 interface for SQLite databases*. Disponible sur : <https://docs.python.org/3/library/sqlite3.html>.
- [4] OWASP Foundation. *OWASP Top Ten, Security Risks*, 2021. Disponible sur : <https://owasp.org/www-project-top-ten>.
- [5] Provos, N. et Mazières, D. *A Future-Adaptable Password Scheme*. USENIX Annual Technical Conference, 1999.
- [6] National Institute of Standards and Technology (NIST). *FIPS 197, Advanced Encryption Standard (AES)*, 2001. Disponible sur : <https://csrc.nist.gov/publications/detail/fips/197/final>.
- [7] NIST. *SP 800-63B, Digital Identity Guidelines : Authentication and Lifecycle Management*, 2017. Disponible sur : <https://pages.nist.gov/800-63-3/sp800-63b.html>.
- [8] Sandhu, R. et Samarati, P. *Access Control : Principles and Practice*. IEEE Communications Magazine, vol. 32, no. 9, pp. 40-48, septembre 1994.
- [9] Nextcloud GmbH. *Nextcloud Documentation*. Disponible sur : <https://docs.nextcloud.com>.
- [10] Fowler, M. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.
- [11] Chacon, S. et Straub, B. *Pro Git*, 2^{ème} édition. Apress, 2014. Disponible sur : <https://git-scm.com/book/fr/v2>.